

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA

Universidad Tecnológica Nacional — Facultad Regional Mendoza

TRABAJO PRÁCTICO INTEGRADOR

Gestión de Datos de Países en Python:

Filtros, Ordenamientos y Estadísticas

Integrantes

Erika Oviedo
Fabiana Rodriguez

Docentes

Prof. Ariel Enferrel
Prof. Cinthia Rigoni

Materia

Programación 1 — 1.º semestre 2026

Fecha de entrega

Junio 2026

Índice

Índice.....	2
1. Introducción.....	3
2. Marco Teórico.....	4
2.1 Listas en Python.....	4
Ejemplo conceptual.....	4
2.2 Diccionarios en Python.....	4
Ejemplo conceptual.....	4
2.3 Funciones.....	4
2.4 Estructuras Condicionales.....	5
Ejemplo conceptual.....	5
2.5 Estructuras Repetitivas.....	5
Ciclo while.....	5
Ciclo for.....	5
2.6 Archivos CSV.....	5
Ejemplo de registro CSV.....	6
2.7 Algoritmo de Ordenamiento — Bubble Sort.....	6
Pseudocódigo simplificado.....	6
2.8 Estadísticas Descriptivas.....	6
2.9 Validación de Datos y Manejo de Errores.....	6
2.10 Modularización del Código.....	7
3. Objetivo del Trabajo.....	8
4. Diseño del Caso Práctico.....	9
4.1 Estructura del Proyecto.....	9
4.2 Estructura de Datos.....	9
4.3 Funcionalidades del Sistema.....	9
4.4 Flujo de Operaciones.....	9
5. Resultados Obtenidos.....	10
5.1 Funcionalidades Implementadas.....	10
5.2 Mejoras Aplicadas respecto a la Versión Inicial.....	10
6. Dificultades y Conclusiones.....	11
6.1 Dificultades Encontradas.....	11
6.2 Conclusiones.....	11
7. Criterios de Evaluación.....	13
8. Bibliografía.....	14
9. Links del Proyecto.....	14

1. Introducción

El presente trabajo práctico integrador tiene como finalidad desarrollar una aplicación de consola en Python para la gestión de información sobre países del mundo. El sistema permite cargar datos desde un archivo CSV, realizar búsquedas, aplicar filtros por distintos criterios, ordenar registros y generar estadísticas descriptivas sobre el dataset.

Para su implementación se aplicaron los conceptos fundamentales de Programación 1: listas, diccionarios, funciones, estructuras condicionales y repetitivas, manejo de archivos CSV, algoritmos de ordenamiento, validación de datos y modularización del código. La correcta integración de estas herramientas permitió construir una solución organizada, escalable y fácil de mantener.

2. Marco Teórico

2.1 Listas en Python

Las listas son estructuras de datos dinámicas que permiten almacenar múltiples elementos dentro de una misma variable. Se caracterizan por ser ordenadas, mutables e indexables, lo que habilita el acceso directo a cualquier elemento mediante su posición (Python Software Foundation, 2024).

En este proyecto, la lista constituye la estructura principal de almacenamiento: todos los países cargados desde el CSV se guardan en una variable denominada `países`. Esta decisión facilita la iteración con ciclos `for`, la aplicación de filtros mediante comprensión de listas y la realización de búsquedas y ordenamientos sobre el conjunto completo de registros.

Ejemplo conceptual

```
países = [
    {"nombre": "Argentina", "poblacion": 45376763},
    {"nombre": "Brasil", "poblacion": 213993437},
]
```

2.2 Diccionarios en Python

Los diccionarios almacenan información mediante pares clave-valor. Cada clave permite acceder directamente a un dato específico sin necesidad de recorrer la estructura completa. Son mutables y no admiten claves duplicadas (Lutz, 2013).

En la aplicación, cada país se representa como un diccionario con cuatro campos: `nombre`, `poblacion`, `superficie` y `continente`. Esta estructura permite acceder a cualquier atributo de forma directa durante las operaciones de búsqueda, filtrado, ordenamiento y generación de estadísticas.

Ejemplo conceptual

```
pais = {
    "nombre": "Argentina",
    "poblacion": 45376763,
    "superficie": 2780400,
    "continente": "América",
}
```

La combinación de listas y diccionarios constituye una solución eficiente para representar datasets tabulares sin requerir bases de datos externas.

2.3 Funciones

Las funciones son bloques de código reutilizables diseñados para realizar tareas específicas. Favorecen la modularización, evitan la repetición de lógica y simplifican el mantenimiento del sistema (Van Rossum & Drake, 2009).

La aplicación sigue el principio de responsabilidad única: cada función realiza exactamente una tarea. Las funciones principales implementadas son:

- `agregar_pais()` — registra un nuevo país con validaciones.
- `actualizar_pais()` — modifica población y/o superficie de un país existente.
- `buscar_pais()` — busca por nombre con coincidencia parcial.
- `filtrar_por_continente()` / `filtrar_por_poblacion()` / `filtrar_por_superficie()`
- `ordenar_burbuja()` — implementa Bubble Sort con soporte ascendente/descendente.
- `mostrar_estadisticas()` — calcula y presenta indicadores descriptivos.

- `cargar_paises()` / `guardar_paises()` — gestionan la persistencia en CSV.

2.4 Estructuras Condicionales

Las estructuras condicionales permiten ejecutar distintos bloques de código según el cumplimiento de una condición. En Python se implementan con `if`, `elif` y `else` (Python Software Foundation, 2024).

En el proyecto se utilizan para:

- Validar que los campos ingresados no estén vacíos ni sean negativos.
- Evitar registros duplicados al agregar un nuevo país.
- Verificar que un rango de filtro sea coherente (mínimo $\leq$ máximo).
- Informar al usuario cuando una búsqueda no arroja resultados.
- Controlar errores de formato al leer el archivo CSV.

Ejemplo conceptual

```
if any(normalizar(p["nombre"]).lower() == nombre.lower() for p in paises):  
    print("Ya existe ese país.")  
    return
```

2.5 Estructuras Repetitivas

Las estructuras repetitivas permiten ejecutar instrucciones múltiples veces. Python ofrece el ciclo `while` para repetición condicionada y el ciclo `for` para iteración sobre secuencias (Lutz, 2013).

Ciclo while

Mantiene activo el menú principal y los submenús de filtros hasta que el usuario decide salir. También se utiliza en las funciones de validación de entrada para repetir la solicitud hasta recibir un dato correcto.

Ciclo for

Se emplea para:

- Recorrer todos los países al mostrar la lista, calcular promedios y estadísticas.
- Aplicar filtros sobre el dataset mediante comprensión de listas.
- Ejecutar el algoritmo Bubble Sort con doble iteración anidada.
- Contar la cantidad de países por continente.

2.6 Archivos CSV

CSV (Comma-Separated Values) es un formato ampliamente utilizado para almacenar información tabular en texto plano. Cada fila representa un registro y cada columna corresponde a un atributo separado por comas (RFC 4180, 2005).

El sistema implementa persistencia mediante el módulo estándar `csv` de Python. Al iniciar, los datos se cargan desde el archivo `datos.csv`; las modificaciones se guardan automáticamente después de cada operación de escritura.

Se implementaron los siguientes mecanismos de validación al leer el CSV:

- Verificación de columnas requeridas: nombre, poblacion, superficie, continente.
- Conversión y validación de tipos de datos (`int` para población y superficie).
- Rechazo de filas con campos vacíos o valores no positivos.

- Manejo de excepciones OSError para errores de I/O.

Ejemplo de registro CSV

```
nombre,poblacion,superficie,continente
Argentina,45376763,2780400,América
Japón,125800000,377975,Asia
Brasil,213993437,8515767,América
```

2.7 Algoritmo de Ordenamiento — Bubble Sort

Bubble Sort (Ordenamiento Burbuja) es un algoritmo de comparación que recorre repetidamente la lista e intercambia elementos adyacentes cuando están en el orden incorrecto. El proceso se repite hasta que no hay más intercambios (Cormen et al., 2009).

Características de la implementación:

- Opera sobre una copia de la lista original; no modifica el dataset en memoria.
- Soporta orden ascendente y descendente mediante el parámetro reverso.
- Compara strings de forma insensible a mayúsculas para el criterio nombre.

Pseudocódigo simplificado

```
resultado = paises[:]
n = len(resultado)

for i in range(n-1):
    for j in range(n - 1 - i):
        valor_actual = resultado[j][criterio]
        valor_siguiente = resultado[j + 1][criterio]

        if reverso:
            hay_que_intercambiar = valor_actual < valor_siguiente
        else:
            hay_que_intercambiar = valor_actual > valor_siguiente
        if hay_que_intercambiar:
            resultado[j], resultado[j + 1] = resultado[j + 1], resultado[j]
```

2.8 Estadísticas Descriptivas

La estadística descriptiva resume información mediante indicadores numéricos que facilitan la comprensión de un conjunto de datos (Wackerly et al., 2010).

El sistema calcula y muestra los siguientes indicadores:

- País con mayor y menor población.
- País con mayor y menor superficie.
- Promedio de población.
- Promedio de superficie.
- Cantidad de países por continente (con barra visual proporcional).

Los valores extremos se obtienen reutilizando la función `ordenar_burbuja()`: el primer elemento de la lista ordenada ascendentemente corresponde al mínimo y el último al máximo. Los promedios se calculan acumulando el total con un ciclo for y dividiendo por la cantidad de registros.

2.9 Validación de Datos y Manejo de Errores

La validación garantiza que la información almacenada sea correcta y consistente. El manejo de excepciones evita que errores inesperados interrumpan la ejecución del programa (Van Rossum & Drake, 2009).

Validaciones implementadas:

- Textos no vacíos: `pedir_texto_no_vacio()` repite la solicitud hasta recibir contenido.
- Enteros positivos: `pedir_entero_positivo()` rechaza ceros, negativos y no numéricos.
- Rangos coherentes: en filtros, se verifica que mínimo $\leq$ máximo.
- Duplicados: al agregar, se compara el nombre con los existentes (case-insensitive).
- CSV: se validan columnas, tipos de datos, vacíos y valores negativos.
- Excepciones `OSError` capturadas en lectura y escritura de archivos.
- Normalizar los nombres de países y continentes que contienen acentos ortográficos: uso de `Unicodedata`.

2.10 Modularización del Código

La modularización consiste en dividir una aplicación en componentes independientes, cada uno con una responsabilidad específica. Facilita el mantenimiento, las pruebas y la reutilización del código (Martin, 2008).

El proyecto se divide en tres módulos:

Módulo	Responsabilidad
main.py	Punto de entrada. Menú principal y coordinación entre módulos.
países.py	Lógica de negocio: agregar, actualizar, buscar, filtrar, ordenar y estadísticas.
archivo_csv.py	Persistencia: lectura y escritura del archivo <code>datos.csv</code> .

3. Objetivo del Trabajo

Desarrollar una aplicación de consola en Python que permita gestionar información sobre países, aplicando listas, diccionarios, funciones, estructuras condicionales y repetitivas, ordenamientos y estadísticas. El sistema debe leer datos desde un archivo CSV, realizar consultas y generar indicadores clave a partir del dataset.

Los objetivos específicos son:

- Implementar un menú interactivo que cubra todas las funcionalidades requeridas.
- Garantizar la integridad de los datos mediante validaciones robustas.
- Aplicar el algoritmo Bubble Sort implementado manualmente.
- Calcular estadísticas descriptivas sobre el conjunto de países.
- Persistir los cambios automáticamente en un archivo CSV.
- Organizar el código en módulos con responsabilidades bien definidas.

4. Diseño del Caso Práctico

4.1 Estructura del Proyecto

```

tp-paises/
├── main.py           # Punto de entrada y menú principal
├── paises.py         # Lógica de negocio
├── archivo_csv.py    # Persistencia CSV
├── datos.csv         # Dataset con 30 países de ejemplo
└── README.md        # Documentación del repositorio

```

4.2 Estructura de Datos

Cada país se representa como un diccionario Python con cuatro campos. Todos los países se almacenan en una lista de diccionarios en memoria.

Campo	Tipo — Restricciones
nombre	str — No vacío, único (case-insensitive)
poblacion	int — Mayor a 0
superficie	int — Mayor a 0 (km²)
continente	str — No vacío

4.3 Funcionalidades del Sistema

Opción	Descripción
1 — Agregar país	Registra un nuevo país con validación completa.
2 — Actualizar país	Modifica población y/o superficie; 0 conserva el valor actual.
3 — Buscar país	Búsqueda parcial por nombre, insensible a mayúsculas.
4 — Filtrar países	Por continente, rango de población o rango de superficie.
5 — Ordenar países	Bubble Sort por nombre, población o superficie (asc/desc).
6 — Ver estadísticas	Extremos, promedios y distribución por continente.
7 — Mostrar todos	Tabla completa de todos los países del dataset.
8 — Salir	Termina el programa.

4.4 Flujo de Operaciones

Al iniciar, main.py llama a cargar_paises() que lee el CSV y retorna la lista de países. El menú principal presenta las opciones con `questionary.select()`. Cada selección delega la ejecución a la función correspondiente de paises.py. Las operaciones de escritura (agregar y actualizar) invocan guardar_paises() automáticamente para persistir los cambios. El ciclo while se repite hasta que el usuario selecciona la opción Salir.

5. Resultados Obtenidos

5.1 Funcionalidades Implementadas

Todas las funcionalidades mínimas requeridas por la consigna fueron implementadas y verificadas:

- Agregar país con validación de duplicados y campos obligatorios.
- Actualizar población y superficie (0 conserva el valor actual).
- Búsqueda parcial por nombre con resultado en tabla.
- Filtros por continente, rango de población y rango de superficie.
- Ordenamiento Bubble Sort por tres criterios en orden ascendente/descendente.
- Estadísticas con extremos, promedios y distribución por continente.
- Persistencia automática en CSV con validación de formato al cargar.

5.2 Mejoras Aplicadas respecto a la Versión Inicial

Área	Mejora implementada
Validación de entrada	pedir_entero_no_negativo() para actualizar sin rechazar el 0.
Unicodedata	Validar nombres de países y continentes con acentos ortográficos.
Menú con Questionary	Menú interactivo en la terminal, muestra una lista navegable mediante las teclas del teclado.
Modularización	Funciones auxiliares privadas _calcular_promedio() y _contar_por_continente().
Separación de módulos	Constante COLUMNAS_REQUERIDAS y función _validar_fila() en archivo_csv.py.
Menú principal	OPCIONES_MENU como constante para facilitar futuras modificaciones.
Consistencia UX	continuar() en todos los casos del menú de filtros; mensajes uniformes con ⚠ y ✓.
Manejo de errores	OSError en lugar de Exception genérico; mensaje de error más específico.

6. Dificultades y Conclusiones

6.1 Dificultades Encontradas

Durante el desarrollo del proyecto se presentaron diversos desafíos técnicos que requirieron ajustes y mejoras en la implementación del sistema.

Uno de los principales inconvenientes estuvo relacionado con el diseño de las funciones de validación de entrada. La funcionalidad de actualización de datos debía permitir que el usuario ingresara el valor **0** como indicador de que no deseaba modificar un campo determinado. Sin embargo, la función de validación utilizada inicialmente rechazaba este valor al considerarlo inválido. Para resolver esta situación, se desarrolló una función específica denominada **pedir_entero_no_negativo()**, capaz de aceptar valores iguales o mayores a cero según los requerimientos de esta operación.

Otro desafío surgió durante la implementación del algoritmo de ordenamiento **Bubble Sort**. Se detectó que la comparación directa de cadenas de texto generaba resultados inconsistentes cuando los nombres de los países presentaban diferencias en el uso de mayúsculas y minúsculas. Para garantizar un ordenamiento correcto y uniforme, se normalizaron los valores convirtiéndolos a minúsculas antes de realizar las comparaciones.

Asimismo, durante la fase de pruebas se identificó una inconsistencia en la validación de nombres de países. El sistema permitía registrar un país ya existente cuando este era ingresado sin sus correspondientes acentos ortográficos, lo que podía generar registros duplicados. Del mismo modo, las funciones de búsqueda de países y filtrado por continente no reconocían correctamente coincidencias cuando el usuario omitía los acentos en los términos ingresados, devolviendo resultados erróneos indicando que dichos elementos no existían.

Para solucionar esta problemática, se incorporó la biblioteca **unicodedata** junto con una función de normalización de texto. Esta implementación permitió eliminar las diferencias derivadas del uso o la omisión de acentos durante las comparaciones, garantizando un comportamiento consistente en las operaciones de alta, búsqueda y filtrado de información. Como resultado, se mejoró significativamente la precisión de las validaciones y la experiencia de uso del sistema.

Finalmente, la coordinación del trabajo en equipo también representó un desafío importante. Fue necesario establecer convenciones claras respecto a la nomenclatura de funciones, la organización de los módulos y el formato de los mensajes de error, con el objetivo de mantener un código homogéneo, coherente y fácil de mantener entre ambas integrantes del grupo.

6.2 Conclusiones

El desarrollo de este trabajo permitió consolidar el uso de las estructuras de datos fundamentales de Python (listas y diccionarios) aplicadas a un problema real de gestión de información. La implementación manual del algoritmo Bubble Sort evidenció la importancia de comprender el funcionamiento interno de los algoritmos antes de recurrir a soluciones de más alto nivel.

La separación del código en módulos con responsabilidades bien definidas facilitó el desarrollo paralelo, redujo los conflictos de integración y produjo un sistema más legible y mantenible. El principio de una función = una responsabilidad demostró su valor en la práctica.

Finalmente, el manejo de archivos CSV y la validación de sus datos reforzaron la comprensión de la persistencia de información y la importancia de anticipar casos borde para construir aplicaciones robustas.

7. Criterios de Evaluación

A continuación se presenta el resumen de cumplimiento de los criterios de evaluación establecidos en la consigna del TPI:

Criterio	Descripción	Estado
Funcionalidad	Búsquedas, filtros, ordenamientos y estadísticas completos.	✓
Estructuras	Uso correcto de listas y diccionarios como dataset principal.	✓
Calidad de código	Modularización, legibilidad, docstrings y comentarios.	✓
Documentación	README claro con ejemplos e informe teórico coherente.	✓
Video	10-15 min mostrando todos los flujos con participación equitativa.	✓
Entrega GitHub	Repositorio con código fuente, informe PDF y archivo CSV.	✓

8. Bibliografía

[Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. \(2009\). Introduction to Algorithms \(3.a ed.\). MIT Press.](#)

Lutz, M. (2013). Learning Python (5.a ed.). O'Reilly Media.

[Martin, R. C. \(2008\). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.](#)

[Python Software Foundation. \(2024\). The Python Tutorial.](#)

[Python Software Foundation. \(2024\). csv — CSV File Reading and Writing.](#)

[RFC 4180. \(2005\). Common Format and MIME Type for Comma-Separated Values \(CSV\) Files.](#)

[Van Rossum, G., & Drake, F. L. \(2009\). The Python Language Reference. Python Software Foundation.](#)

[Wackerly, D. D., Mendenhall, W., & Scheaffer, R. L. \(2010\). Mathematical Statistics with Applications \(7.a ed.\). Cengage Learning.](#)

9. Links del Proyecto

[Repositorio en GitHub](#)

[Video demostrativo \(YouTube / Drive\)](#)

[Presentación](#)